数据库与 SQL 高频笔试题、面试题及答案

> 适用目标：自动化测试工程师、测试开发工程师、Python 测试平台、FastAPI、Jenkins、5G KPI 数据分析相关岗位。
> 回答风格建议：先说明数据库概念，再结合 SQL 示例，最后联系"测试运行记录、KPI、artifact、callback、RCA、PostgreSQL / SQLite"等项目场景。

第一部分：数据库高频面试题

1. 什么是数据库、DBMS 和关系型数据库？

回答：

- 数据库：按照一定结构保存的数据集合。
- DBMS：用于创建、查询、更新、备份和管理数据库的软件，例如 PostgreSQL、MySQL、SQLite。
- 关系型数据库：使用表、行、列表示数据，并通过主键、外键和 SQL 建立关系。

测试平台中，run、stage、KPI、artifact 和 RCA revision 都具有明确关系，因此适合使用关系型数据库。

2. 关系型数据库和 NoSQL 数据库有什么区别？

回答：

关系型数据库：

- Schema 明确。
- 支持 JOIN 和事务。
- 强调数据一致性。
- 适合订单、测试运行、状态机、审计等结构化数据。

NoSQL：

- 数据模型更灵活。
- 常用于文档、缓存、海量日志或高吞吐场景。
- 不同产品的一致性和查询能力差异较大。

当前 5G KPI 平台第一期选择 PostgreSQL，因为既需要事务和关联查询，也可以使用 JSONB 保存扩展字段，不需要额外引入 MongoDB。

3. 主键、唯一约束和外键有什么区别？

回答：

- Primary Key：唯一标识一行，不能为 NULL；一张表只有一个主键约束，但可以是联合主键。
- Unique：保证指定列组合不重复，表中可以有多个唯一约束。
- Foreign Key：保证引用的数据存在，维护表之间的引用完整性。

示例：

```
```

```sql
CREATE TABLE test_runs (
    run_id UUID PRIMARY KEY,
    external_request_id VARCHAR(100) UNIQUE,
    status VARCHAR(30) NOT NULL
);

CREATE TABLE execution_stages (
    stage_id BIGSERIAL PRIMARY KEY,
    run_id UUID NOT NULL REFERENCES test_runs(run_id),
    stage_name VARCHAR(100) NOT NULL
);
```

4. 什么是数据库三范式？

回答：

- 第一范式：字段保持原子性，不在一个字段中保存多个独立值。
- 第二范式：在第一范式基础上，非主键字段完全依赖整个主键。
- 第三范式：在第二范式基础上，非主键字段不依赖其他非主键字段。

范式可以减少重复和更新异常，但真实项目也会适度反范式。例如 KPI summary 可以保存部分已计算统计值，避免每次页面查询都重新扫描原始 KPI 文件。

5. 什么是反范式？什么时候使用？

回答：

反范式是为了查询性能或使用便利，有意识地保存部分冗余数据。

适用情况：

- 数据读取远多于更新。
- 聚合计算成本高。
- 页面需要快速展示摘要。
- 冗余字段可以通过程序或约束保证一致。

例如 `test_runs` 可以保存最终 `anomaly_count`，但原始 anomaly 记录仍然存放在 anomaly 表。不能为了少写一次 JOIN 就无控制地复制所有数据。

6. SQL 中 DDL、DML、DQL、DCL、TCL 分别是什么？

回答：

- DDL：定义结构，例如 `CREATE`、`ALTER`、`DROP`。
- DML：修改数据，例如 `INSERT`、`UPDATE`、`DELETE`。
- DQL：查询数据，主要是 `SELECT`。
- DCL：权限控制，例如 `GRANT`、`REVOKE`。
- TCL：事务控制，例如 `COMMIT`、`ROLLBACK`、`SAVEPOINT`。

7. CHAR、VARCHAR 和 TEXT 有什么区别？

回答：

- `CHAR(n)`：固定长度，适合真正固定长度的数据。
- `VARCHAR(n)`：可变长度，并限制最大长度。
- `TEXT`：可变长度文本，通常不设置业务长度上限。

在 PostgreSQL 中，`VARCHAR` 和 `TEXT` 性能通常没有本质差异，主要根据业务约束选择。状态、类型适合 `VARCHAR`；长日志摘要和报告适合 `TEXT`。

8. NULL 表示什么？查询时有什么注意点？

回答：

NULL 表示未知、缺失或不适用，不等于空字符串、0 或 False。

错误写法：

```
WHERE finished_at = NULL
```

正确写法：

```
WHERE finished_at IS NULL
```

常用：

```
SELECT COALESCE(error_message, 'no error')
FROM test_runs;
```

聚合函数通常忽略 NULL，但 `COUNT(*)` 统计所有行，`COUNT(column)` 只统计该列非 NULL 的行。

9. WHERE 和 HAVING 有什么区别？

回答：

- `WHERE` 在分组前过滤原始行。
- `HAVING` 在 `GROUP BY` 后过滤聚合结果。

```
SELECT testline, COUNT(*) AS failed_count
FROM test_runs
WHERE status = 'failed'
GROUP BY testline
HAVING COUNT(*) >= 5;
```

能放在 WHERE 的条件优先放 WHERE，可以减少进入分组的数据量。

10. INNER JOIN、LEFT JOIN、RIGHT JOIN 和 FULL JOIN 有什么区别？

回答：

- INNER JOIN：只返回两边匹配的数据。
- LEFT JOIN：返回左表所有数据，右表无匹配时为 NULL。
- RIGHT JOIN：返回右表所有数据。
- FULL JOIN：返回两边所有数据。

查找没有 KPI 结果的 run，可以使用 LEFT JOIN：

```
SELECT r.run_id
FROM test_runs r
LEFT JOIN kpi_results k ON k.run_id = r.run_id
WHERE k.run_id IS NULL;
```

实际项目中更常使用 INNER JOIN 和 LEFT JOIN。

11. JOIN 和子查询如何选择？

回答：

没有绝对规则：

- 需要同时返回多张表字段时，JOIN 更直观。
- 判断是否存在时，`EXISTS` 通常表达更清楚。
- 复杂逻辑可以用 CTE 分阶段表达。
- 最终性能应结合执行计划，而不是只看 SQL 写法。

示例：

```
SELECT r.run_id
FROM test_runs r
WHERE EXISTS (
    SELECT 1
    FROM anomalies a
    WHERE a.run_id = r.run_id
      AND a.severity = 'critical'
);
```

12. UNION 和 UNION ALL 有什么区别？

回答：

- `UNION` 合并结果后去重。
- `UNION ALL` 保留重复数据，通常更快。

如果业务上允许重复或已经确认两组数据互斥，优先使用 `UNION ALL`，避免不必要的排序或哈希去重。

13. DELETE、TRUNCATE 和 DROP 有什么区别？

回答：

- `DELETE`：按条件删除行，可以触发行级约束和触发器。
- `TRUNCATE`：快速清空整张表，不能使用 WHERE，锁和事务行为取决于数据库；PostgreSQL 中可事务回滚。
- `DROP`：删除表结构和数据。

生产环境不能因为“清空测试数据”就直接使用 TRUNCATE 或 DROP。需要确认外键、权限、备份和目标环境。

14. GROUP BY 有什么作用？

回答：

`GROUP BY` 按一个或多个字段分组，再配合聚合函数计算：

- `COUNT`
- `SUM`
- `AVG`
- `MIN`
- `MAX`

```
SELECT status, COUNT(*) AS run_count
FROM test_runs
GROUP BY status
ORDER BY run_count DESC;
```

SELECT 中没有聚合的普通字段通常必须出现在 GROUP BY 中。

15. 子查询和 CTE 有什么区别？

回答：

子查询嵌套在 SELECT、FROM 或 WHERE 中。CTE 使用 `WITH` 定义命名的中间结果，可读性更好，也支持递归。

```
WITH failed_runs AS (
    SELECT run_id, testline
    FROM test_runs
    WHERE status = 'failed'
)
SELECT testline, COUNT(*)
FROM failed_runs
GROUP BY testline;
```

CTE 不一定比子查询快，仍需查看执行计划。现代 PostgreSQL 可以对部分 CTE 进行内联优化。

16. 什么是窗口函数？

回答：

窗口函数在保留明细行的同时，对相关数据进行排名、累计或前后行比较。

常见：

- `ROW_NUMBER`
- `RANK`
- `DENSE_RANK`
- `LAG`
- `LEAD`
- `SUM() OVER`
- `AVG() OVER`

```
SELECT
    run_id,
    testline,
    started_at,
    ROW_NUMBER() OVER (
        PARTITION BY testline
        ORDER BY started_at DESC
    ) AS rn
FROM test_runs;
```

窗口函数非常适合查询每条测试线最近一次运行、KPI 趋势和环比变化。

17. ROW_NUMBER、RANK 和 DENSE_RANK 有什么区别？

回答：

假设成绩为 100、90、90、80：

- `ROW_NUMBER`：1、2、3、4。
- `RANK`：1、2、2、4。
- `DENSE_RANK`：1、2、2、3。

如果要求每组严格取三行，用 `ROW_NUMBER`；如果并列值应该共享名次，用 `RANK` 或 `DENSE_RANK`。

18. View 和 Materialized View 有什么区别？

回答：

- View：保存查询定义，每次查询时读取基础表。
- Materialized View：保存查询结果，查询快，但数据不会自动实时更新，需要 refresh。

KPI 趋势查询复杂且允许一定延迟时，可以使用 Materialized View；run 当前状态应直接查询基础表。

19. 什么是事务？ACID 分别代表什么？

回答：

事务是一组不可分割的数据库操作。

- Atomicity：原子性，要么全部成功，要么全部回滚。
- Consistency：一致性，事务前后满足约束。
- Isolation：隔离性，并发事务尽量互不干扰。
- Durability：持久性，提交后结果可以恢复。

例如 callback 同时更新 run 终态、stage、artifact 和 KPI summary，应在一个事务中完成，避免只写入一部分。

20. 常见事务隔离级别有哪些？

回答：

从低到高：

- Read Uncommitted
- Read Committed
- Repeatable Read
- Serializable

常见并发问题：

- 脏读：读到未提交数据。
- 不可重复读：同一事务两次读取同一行结果不同。
- 幻读：同一条件两次查询出现新增或消失的行。

PostgreSQL 默认是 Read Committed。隔离级别越高并发冲突可能越多，应根据业务一致性要求选择。

21. 什么是 MVCC？

回答：

MVCC 是多版本并发控制。数据库为数据保留不同版本，使读操作通常不需要阻塞写操作。

PostgreSQL 使用 MVCC 提高并发，但旧版本需要通过 VACUUM 回收。如果长事务一直不结束，可能阻止旧版本清理并造成表膨胀。

22. 什么是数据库锁？常见锁有哪些？

回答：

数据库通过锁保证并发一致性。

常见：

- 行锁：锁定部分数据行。
- 表锁：锁定整张表或表级操作。
- 共享锁：允许其他事务读取。
- 排他锁：阻止冲突修改。

PostgreSQL 中可以使用：

```
```

```sql
SELECT *
FROM analysis_jobs
WHERE status = 'queued'
ORDER BY created_at
FOR UPDATE SKIP LOCKED
LIMIT 1;
```

多个 Worker 可以安全领取不同任务，而不会重复处理同一条记录。

23. 什么是死锁？如何避免？

回答：

死锁是两个事务互相等待对方持有的锁。

避免方式：

- 多个事务按相同顺序访问表和行。
- 缩短事务时间。
- 不在事务中执行长时间网络请求。
- 合理建立索引，减少锁定范围。
- 捕获 deadlock 异常并对幂等事务做有限重试。

数据库会检测死锁并终止其中一个事务，所以应用还需要正确处理异常。

24. 乐观锁和悲观锁有什么区别？

回答：

- 悲观锁：假设冲突容易发生，先锁定数据，例如 `SELECT ... FOR UPDATE`。
- 乐观锁：假设冲突较少，更新时通过 version 或时间戳检查数据是否被修改。

示例：

```sql
UPDATE test_runs
SET status = 'running',
    version = version + 1
WHERE run_id = :run_id
  AND version = :expected_version;
```

如果受影响行数为 0，说明发生并发修改。

25. 什么是数据库索引？

回答：

索引是帮助数据库快速定位数据的数据结构，类似书的目录。

适合建立索引的字段：

- 主键和唯一键。
- JOIN 字段。
- 高频 WHERE 条件。
- 排序和范围查询字段。

代价：

- 占用磁盘。
- INSERT、UPDATE、DELETE 需要维护索引。
- 索引过多会降低写入性能。

26. 常见索引类型有哪些？

回答：

PostgreSQL 常见：

- B-tree：默认索引，适合等值、范围和排序。
- Hash：主要用于等值查询。
- GIN：适合 JSONB、数组和全文检索。
- GiST：适合范围、空间和特定复杂类型。
- BRIN：适合超大且物理顺序与字段相关的数据。

第一期测试平台大部分字段使用 B-tree；RCA 文本检索可以使用全文索引和 `pg_trgm`。

27. 什么是联合索引和最左前缀原则？

回答：

联合索引包含多个字段：

```
CREATE INDEX idx_runs_testline_created
ON test_runs(testline, created_at DESC);
```

它适合：

```
WHERE testline = ?
ORDER BY created_at DESC
```

一般可以从索引最左侧字段开始使用。如果只按 `created_at` 查询，这个索引可能不能充分发挥作用。

PostgreSQL 是否使用索引仍由优化器根据统计信息和查询成本决定，不能机械套用规则。

28. 哪些情况下数据库可能不使用索引？

回答：

- 表很小，全表扫描更快。

- 查询返回大部分数据。
- 对索引列进行了不匹配的函数或类型转换。
- 使用前导通配符，例如 `LIKE '%error'`。
- 联合索引没有匹配合适的前导列。
- 统计信息过期。
- 字段选择性太低。
- 隐式类型转换。

应使用 `EXPLAIN` 或 `EXPLAIN ANALYZE` 查看真实执行计划。

29. 如何排查慢 SQL？

回答：

步骤：

1. 获取 SQL、参数、耗时和调用频率。
2. 使用 `EXPLAIN (ANALYZE, BUFFERS)`。
3. 检查是否全表扫描、估算偏差、排序、临时文件或重复循环。
4. 检查索引和统计信息。
5. 减少无用列、行和重复查询。
6. 检查 JOIN 条件及数据类型。
7. 判断是否存在 N+1 查询。
8. 优化后重新测量。

`EXPLAIN ANALYZE` 会真实执行 SQL，不能对生产环境中的高风险 UPDATE/DELETE 随意使用。

30. 什么是 SQL 注入？如何防止？

回答：

SQL 注入是将用户输入拼接进 SQL，使攻击者改变原本语义。

错误：

```
sql = f"SELECT * FROM users WHERE name = '{name}'"
```

正确：

```
cursor.execute(
    "SELECT * FROM users WHERE name = %s",
    (name,),
)
```

防护：

- 参数化查询。

- ORM。
- 输入校验。
- 最小数据库权限。
- 不把任意字段名、排序表达式或 SQL 片段直接交给用户。

31. 什么是数据库连接池？

回答：

连接池复用已有数据库连接，避免每个请求都重新建立 TCP、认证和会话。

需要配置：

- pool size。
- max overflow。
- 获取连接超时。
- 连接回收和健康检查。

连接池不是越大越好。总连接数必须结合 PostgreSQL `max_connections`、API Worker 数量和数据库资源计算。

32. PostgreSQL 和 SQLite 有什么区别？

回答：

SQLite：

- 嵌入式单文件。
- 不需要数据库服务。
- 部署简单。
- 适合本地工具、原型和低并发场景。
- 并发写入能力有限。

PostgreSQL：

- 独立数据库服务。
- 支持多连接并发、事务、权限、备份和丰富索引。
- 支持 JSONB、全文检索和复杂查询。
- 更适合真实测试平台。

旧项目使用 SQLite 快速完成 MVP；新 5G KPI 平台从第一期使用 PostgreSQL，避免 callback、Worker 和 API 并发写入时遇到 SQLite 锁竞争，也避免后续迁移成本。

33. PostgreSQL 中 JSON 和 JSONB 有什么区别？

回答：

- JSON：主要保存原始文本格式。
- JSONB：保存解析后的二进制结构，便于查询和建立 GIN 索引。

适合 JSONB：

- 请求参数快照。
- 可扩展 metadata。
- RCA 结构化补充信息。

不适合全部塞进 JSONB：

- 高频查询和 JOIN 的核心字段。
- 需要外键和强约束的数据。

核心字段应使用正常列，变化较大的扩展信息再使用 JSONB。

34. 什么是数据库 migration？为什么需要 Alembic？

回答：

Migration 是数据库 schema 的版本化变更，例如新增表、字段、索引和约束。

Alembic 可以：

- 保存每次 schema 变化。
- 从空数据库升级到最新版本。
- 在不同环境保持一致。
- 记录当前 revision。
- 在可支持时执行 downgrade。

不能只在数据库中手工加字段，因为新服务器重建时无法知道完整结构。

35. 如何做数据库备份与恢复？

回答：

PostgreSQL 常用：

- `pg_dump`：逻辑备份单个数据库。
- `pg_restore`：恢复自定义格式备份。
- `pg_dumpall`：导出全局对象或多个数据库。
- 物理备份和 WAL：适合更严格的时间点恢复。

测试平台第一期可以：

- 每日逻辑备份。
- migration 前即时备份。
- 备份加密并保存到另一位置。
- 定期恢复到隔离数据库。
- 检查关键表、记录数和查询。

未验证恢复能力的备份不能算真正可用。

36. 如何保证 callback 或消息处理幂等？

回答：

方法：

- 每个 callback 使用唯一 `callback_event_id`。
- 数据库建立唯一约束。
- 状态机阻止非法回退。
- 使用事务。
- PostgreSQL 使用 `INSERT ... ON CONFLICT`。

```
INSERT INTO callback_events (
    callback_event_id,
    run_id,
    event_type,
    occurred_at
)
VALUES (
    :event_id,
    :run_id,
    :event_type,
    :occurred_at
)
ON CONFLICT (callback_event_id) DO NOTHING;
```

应用还要检查受影响行数，不能只依赖“没有抛异常”。

37. 数据库测试应该覆盖哪些内容？

回答：

- 表、字段、类型和默认值。
- 主键、唯一约束和外键。
- CRUD。
- 事务提交与回滚。
- 并发 callback。
- 状态机。
- migration from empty DB。
- 从上一版本升级。
- 备份恢复。
- 索引和关键查询性能。
- 权限和敏感字段。
- 数据保留与清理。

测试 PostgreSQL 特性时不能只用 SQLite 替代，因为并发、JSON、锁和索引行为不同。

第二部分：SQL 高频笔试题

以下题目默认使用 PostgreSQL。

示例表

```sql
CREATE TABLE test_runs (
    run_id UUID PRIMARY KEY,
    testline VARCHAR(100) NOT NULL,
    build_version VARCHAR(100) NOT NULL,
    status VARCHAR(30) NOT NULL,
    requested_by VARCHAR(100),
    started_at TIMESTAMPTZ,
    finished_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE kpi_results (
    kpi_result_id BIGSERIAL PRIMARY KEY,
    run_id UUID NOT NULL REFERENCES test_runs(run_id),
    kpi_name VARCHAR(100) NOT NULL,
    kpi_value NUMERIC,
    unit VARCHAR(30),
    is_anomaly BOOLEAN NOT NULL DEFAULT FALSE,
    collected_at TIMESTAMPTZ NOT NULL
);

CREATE TABLE execution_stages (
    stage_id BIGSERIAL PRIMARY KEY,
    run_id UUID NOT NULL REFERENCES test_runs(run_id),
    stage_name VARCHAR(100) NOT NULL,
    status VARCHAR(30) NOT NULL,
    duration_seconds NUMERIC,
    started_at TIMESTAMPTZ
);
```

38. 查询最近 7 天失败的测试运行

答案：

```sql
SELECT
    run_id,
    testline,
    build_version,
    started_at,
    finished_at
FROM test_runs
WHERE status = 'failed'
  AND created_at >= NOW() - INTERVAL '7 days'
ORDER BY created_at DESC;
```

39. 统计每种状态的运行数量

答案：

```sql
SELECT
    status,
    COUNT(*) AS run_count
FROM test_runs
GROUP BY status
ORDER BY run_count DESC;
```

如果需要显示占比：

```sql
SELECT
    status,
    COUNT(*) AS run_count,
    ROUND(
        COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (),
        2
    ) AS percentage
FROM test_runs
GROUP BY status;
```

40. 查询耗时最长的 5 次运行

答案：

```sql
SELECT
    run_id,
    testline,
    finished_at - started_at AS duration
FROM test_runs
WHERE started_at IS NOT NULL
  AND finished_at IS NOT NULL
ORDER BY duration DESC
LIMIT 5;
```

41. 查询没有任何 KPI 结果的 run

答案一：LEFT JOIN

```sql
SELECT r.run_id, r.testline
FROM test_runs r
LEFT JOIN kpi_results k ON k.run_id = r.run_id
WHERE k.run_id IS NULL;
```

答案二：NOT EXISTS

```sql
SELECT r.run_id, r.testline
FROM test_runs r
WHERE NOT EXISTS (
    SELECT 1
    FROM kpi_results k
    WHERE k.run_id = r.run_id
);
```

`NOT EXISTS` 通常更清楚地表达“没有关联记录”。

42. 统计每个 run 的 KPI 总数和异常数

答案：

```sql
SELECT
    r.run_id,
    COUNT(k.kpi_result_id) AS kpi_count,
    COUNT(k.kpi_result_id) FILTER (
        WHERE k.is_anomaly
    ) AS anomaly_count
FROM test_runs r
LEFT JOIN kpi_results k ON k.run_id = r.run_id
GROUP BY r.run_id
ORDER BY r.created_at DESC;
```

PostgreSQL 的 `FILTER` 可以让条件聚合更清晰。

43. 查询每条 testline 最近一次运行

答案一：窗口函数

```sql
WITH ranked_runs AS (
    SELECT
        r.*,
        ROW_NUMBER() OVER (
            PARTITION BY testline
            ORDER BY created_at DESC
        ) AS rn
    FROM test_runs r
)
SELECT *
FROM ranked_runs
WHERE rn = 1;
```

答案二：PostgreSQL DISTINCT ON

```sql

```

```
SELECT DISTINCT ON (testline)
    *
FROM test_runs
ORDER BY testline, created_at DESC;
```

窗口函数可移植性更好；`DISTINCT ON` 是 PostgreSQL 特性。

44. 查询每条 testline 最近 3 次运行

答案：

```
WITH ranked_runs AS (
    SELECT
        r.*,
        ROW_NUMBER() OVER (
            PARTITION BY testline
            ORDER BY created_at DESC
        ) AS rn
    FROM test_runs r
)
SELECT *
FROM ranked_runs
WHERE rn <= 3
ORDER BY testline, created_at DESC;
```

45. 查询某个 KPI 相比上一次运行的变化

答案：

```
WITH kpi_trend AS (
    SELECT
        r.testline,
        r.run_id,
        r.created_at,
        k.kpi_name,
        k.kpi_value,
        LAG(k.kpi_value) OVER (
            PARTITION BY r.testline, k.kpi_name
            ORDER BY r.created_at
        ) AS previous_value
    FROM test_runs r
    JOIN kpi_results k ON k.run_id = r.run_id
    WHERE k.kpi_name = 'dl_throughput'
)
SELECT
    *,
    kpi_value - previous_value AS absolute_change,
    ROUND(
```

```
        (kpi_value - previous_value)
        / NULLIF(previous_value, 0) * 100,
        2
    ) AS change_percentage
FROM kpi_trend
ORDER BY testline, created_at;
```

`NULLIF(previous_value, 0)` 用来避免除零。

46. 查询平均运行耗时超过 30 分钟的 testline

答案：

```
SELECT
    testline,
    AVG(finished_at - started_at) AS avg_duration
FROM test_runs
WHERE started_at IS NOT NULL
  AND finished_at IS NOT NULL
GROUP BY testline
HAVING AVG(finished_at - started_at) > INTERVAL '30 minutes'
ORDER BY avg_duration DESC;
```

47. 查询连续失败次数较多的 testline

答案：

下面查询每条测试线从最近一次运行开始连续失败的次数：

```
WITH ordered_runs AS (
    SELECT
        testline,
        status,
        created_at,
        SUM(
            CASE WHEN status <> 'failed' THEN 1 ELSE 0 END
        ) OVER (
            PARTITION BY testline
            ORDER BY created_at DESC
        ) AS success_group
    FROM test_runs
),
current_failure_streak AS (
    SELECT
        testline,
        COUNT(*) AS consecutive_failures
    FROM ordered_runs
    WHERE success_group = 0
      AND status = 'failed'
```

```
    GROUP BY testline
)
SELECT *
FROM current_failure_streak
ORDER BY consecutive_failures DESC;
```

面试中如果时间有限，可以先说明使用窗口函数按 testline 排序并划分连续区间。

48. 找出 callback event 表中的重复 event_id

假设表为：

```
callback_events(
    id,
    callback_event_id,
    received_at
)
```

答案：

```
SELECT
    callback_event_id,
    COUNT(*) AS duplicate_count
FROM callback_events
GROUP BY callback_event_id
HAVING COUNT(*) > 1;
```

更重要的是为 `callback_event_id` 建立唯一约束，从源头防止重复。

49. 删除重复记录，每组只保留最新一条

答案：

```
WITH ranked_events AS (
    SELECT
        id,
        ROW_NUMBER() OVER (
            PARTITION BY callback_event_id
            ORDER BY received_at DESC, id DESC
        ) AS rn
    FROM callback_events
)
DELETE FROM callback_events
WHERE id IN (
    SELECT id
    FROM ranked_events
```

```
    WHERE rn > 1
);
```

执行删除前应该先把 DELETE 改成 SELECT，核对目标记录并确认备份。

50. 将超过两小时仍为 running 的任务标记为超时

答案：

```
UPDATE test_runs
SET status = 'infrastructure_failed',
    finished_at = NOW()
WHERE status = 'running'
  AND started_at < NOW() - INTERVAL '2 hours';
```

真实系统还应记录 `status_reason`、审计事件并确认 Jenkins 是否仍在执行，不能只更新数据库而不处理实际任务。

51. 使用事务创建 run 和初始 stages

答案：

```
BEGIN;

INSERT INTO test_runs (
    run_id,
    testline,
    build_version,
    status
)
VALUES (
    :run_id,
    :testline,
    :build_version,
    'created'
);

INSERT INTO execution_stages (
    run_id,
    stage_name,
    status
)
VALUES
    (:run_id, 'precheck', 'pending'),
    (:run_id, 'ue_attach', 'pending'),
    (:run_id, 'dl_traffic', 'pending'),
    (:run_id, 'kpi_collection', 'pending');

COMMIT;
```

任一步失败应 ROLLBACK，避免只有 run 而没有阶段。

52. 使用 PostgreSQL UPSERT 更新 KPI 结果

首先建立唯一约束：

```
ALTER TABLE kpi_results
ADD CONSTRAINT uq_run_kpi_time
UNIQUE (run_id, kpi_name, collected_at);
```

答案：

```
INSERT INTO kpi_results (
    run_id,
    kpi_name,
    kpi_value,
    unit,
    is_anomaly,
    collected_at
)
VALUES (
    :run_id,
    :kpi_name,
    :kpi_value,
    :unit,
    :is_anomaly,
    :collected_at
)
ON CONFLICT (run_id, kpi_name, collected_at)
DO UPDATE SET
    kpi_value = EXCLUDED.kpi_value,
    unit = EXCLUDED.unit,
    is_anomaly = EXCLUDED.is_anomaly;
```

是否允许更新已存在 KPI，需要由业务契约决定；审计型数据也可能选择 `DO NOTHING`。

53. 如何实现大数据量下的稳定分页？

OFFSET 分页：

```
SELECT *
FROM test_runs
ORDER BY created_at DESC, run_id DESC
LIMIT 20 OFFSET 10000;
```

页数较深时数据库需要跳过大量记录。

Keyset 分页：

```
SELECT *
FROM test_runs
WHERE (created_at, run_id) < (:last_created_at, :last_run_id)
ORDER BY created_at DESC, run_id DESC
LIMIT 20;
```

Keyset 分页性能和稳定性更好，但不适合直接跳到任意页。

第三部分：经典通用 SQL 笔试题

假设：

```
CREATE TABLE departments (
    department_id INTEGER PRIMARY KEY,
    department_name VARCHAR(100) NOT NULL
);

CREATE TABLE employees (
    employee_id INTEGER PRIMARY KEY,
    employee_name VARCHAR(100) NOT NULL,
    department_id INTEGER REFERENCES departments(department_id),
    salary NUMERIC NOT NULL,
    email VARCHAR(200)
);
```

54. 查询第二高的工资

答案一：MAX 子查询

```
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (
    SELECT MAX(salary)
    FROM employees
);
```

答案二：DENSE_RANK

```
WITH ranked_salary AS (
    SELECT
        salary,
        DENSE_RANK() OVER (
```

```
            ORDER BY salary DESC
        ) AS salary_rank
    FROM employees
)
SELECT DISTINCT salary
FROM ranked_salary
WHERE salary_rank = 2;
```

如果工资只有一个不同值，结果为空。

55. 查询每个部门工资最高的员工

答案：

```
WITH ranked_employees AS (
    SELECT
        e.*,
        DENSE_RANK() OVER (
            PARTITION BY department_id
            ORDER BY salary DESC
        ) AS salary_rank
    FROM employees e
)
SELECT
    employee_id,
    employee_name,
    department_id,
    salary
FROM ranked_employees
WHERE salary_rank = 1;
```

使用 `DENSE_RANK` 可以保留并列最高工资的员工。

56. 查询每个部门工资最高的前三名员工

答案：

```
WITH ranked_employees AS (
    SELECT
        e.*,
        DENSE_RANK() OVER (
            PARTITION BY department_id
            ORDER BY salary DESC
        ) AS salary_rank
    FROM employees e
)
SELECT
    employee_id,
    employee_name,
```

```
    department_id,
    salary,
    salary_rank
FROM ranked_employees
WHERE salary_rank <= 3
ORDER BY department_id, salary_rank;
```

如果题目要求每部门严格三个人，应改用 ROW_NUMBER 并指定稳定的次级排序字段。

57. 查询重复的邮箱

答案：

```
SELECT
    email,
    COUNT(*) AS email_count
FROM employees
WHERE email IS NOT NULL
GROUP BY email
HAVING COUNT(*) > 1;
```

如果邮箱业务上不允许重复，应增加唯一约束，而不是依赖定期查询。

58. 查询工资高于本部门平均工资的员工

答案：

```
SELECT
    employee_id,
    employee_name,
    department_id,
    salary
FROM (
    SELECT
        e.*,
        AVG(salary) OVER (
            PARTITION BY department_id
        ) AS department_avg_salary
    FROM employees e
) ranked
WHERE salary > department_avg_salary;
```

第四部分：结合当前项目的数据库面试表达

59. 如何设计 5G KPI 测试平台的数据模型？

推荐回答：

我会把结构化状态和大文件分开：

PostgreSQL 保存：

- `test_run`
- `execution_stage`
- `resource_lease`
- `artifact` descriptor
- `kpi_summary`
- `anomaly`
- `analysis_job`
- `analysis_revision`
- `knowledge_case`
- `deployment_event`
- `audit_event`

Jenkins artifact 或文件存储保存：

- TAF 和 traffic 日志。
- KPI Excel / CSV / Parquet。
- HTML 报告。
- Evidence Bundle。

这样既能通过数据库快速查询状态和趋势，也避免把大文件二进制直接塞进 PostgreSQL。

60. 如果面试官问：你在数据库方面最有价值的经验是什么？

推荐回答：

我的数据库经验主要来自自动化测试平台建设，而不是单纯写 CRUD。

我使用数据库保存测试 run、stage、artifact、KPI summary 和 RCA revision；通过事务保证 callback 结果的一致性，通过唯一约束和 UPSERT 保证幂等，通过索引优化 run list、testline 历史和知识案例查询。同时我会把原始日志和 KPI 文件作为 artifact 保存，只把可查询的结构化摘要放入数据库。

旧项目为了快速 MVP 使用 SQLite；重新设计的双服务器 5G KPI 平台选择 PostgreSQL，重点考虑 API、Jenkins callback 和 Analysis Worker 的并发写入、JSONB、全文检索、备份恢复及后续趋势分析。

结合当前简历的数据库面试表达总结

你的简历相关亮点

- 使用 SQLite 完成过测试平台 MVP 数据持久化。
- 理解 PostgreSQL 在真实平台中的并发和事务优势。
- 能设计 run、stage、artifact、KPI、anomaly 和 RCA 数据关系。
- 理解 callback 幂等、唯一约束和状态机。
- 能使用 JOIN、聚合、CTE 和窗口函数分析测试结果。
- 了解索引、执行计划、慢 SQL 和连接池。
- 具备 migration、备份恢复和灾备重建意识。
- 能使用 pytest 验证 repository、事务和 API 数据一致性。

面试时可以这样概括

> 我的数据库经验主要围绕自动化测试平台。旧项目中我使用 SQLite 保存 run、Jenkins build reference、artifact、KPI summary 和 AI analysis，完成了轻量级 MVP。新平台设计中改用 PostgreSQL，重点解决 API、Jenkins callback 和 RCA Worker 并发写入的问题。我会通过外键、唯一约束、事务和状态机保证数据一致性，通过索引优化 run list、testline 历史和 KPI 查询；大体积日志、Excel 和报告仍保存在 artifact storage，数据库只保存 descriptor 和结构化摘要。同时使用 Alembic 管理 schema 版本，并设计 pg_dump、恢复验证和服务器迁移流程。

建议重点准备的个人案例

1. Callback 重复发送时如何通过唯一约束保证幂等。
2. 一次 run、多个 stages 和 artifacts 如何设计表关系。
3. 为什么旧项目用 SQLite、新项目改用 PostgreSQL。
4. 如何查询每条 testline 最近一次运行和 KPI 趋势。
5. 如何通过事务避免 run 终态与 artifact/KPI 只写入一部分。
6. 如何为 run list、testline 历史和 knowledge case 建索引。
7. 如何使用 Alembic、pg_dump 和 restore test 支持环境重建。

数据库问题回答模板

```
1. 先回答概念和适用场景。
2. 给出简短 SQL 或数据模型。
3. 说明一致性、性能和异常情况。
4. 最后结合测试平台中的 run / KPI / callback。
```

示例：

> 比如事务，概念上是一组需要全部成功或全部失败的数据库操作。在测试平台中，Jenkins final callback 会同时更新 run 终态、stage、artifact manifest 和 KPI summary。我会把这些写入放在同一个事务里；如果 KPI 数据校验失败则整体回滚，避免 Portal 看到 run 已完成但查询不到结果。同时 callback_event_id 建唯一约束，保证 Jenkins 重试 callback 时不会重复写入。